Objectives

- To construct a durable, custom handheld console structure using a spacer mounted design for structural integrity.
- Interfacing different electronic components with ESP32 microcontroller.
- To implement the complete algorithms for classic arcade games, specifically Snake game, to ensure real time performance without input lag.
- To develop power system using rechargeable batteries, to ensure the device is truly portable.

Introduction

The evolution of embedded systems has allowed for complex interactive applications to be run-on low-cost hardware. This project focuses on the design and fabrication of a handheld gaming console powered by a microcontroller. While modern gaming consoles rely on high end processors and vast memory resources, this project explores the theoretical and practical limits of implementing real-time gaming on resource-constrained hardware.

Microcontrollers serve as the brain of modern interactive projects. The theoretical core of this project relies on the **ESP32**, a powerful System on Chip (SoC) developed by Espressif Systems. Unlike standard 8-bit microcontrollers (like the Arduino Uno's ATmega328P), the ESP32 features a dual core processor and integrated Wi-Fi/Bluetooth capabilities, making it theoretically ideal for handling the concurrent tasks required in a game console: graphics rendering, input polling, and logic processing.

3. <u>Hardware Components and Design:</u>

The system architecture is divided into four main subsystems: the Central Controller, Output Subsystem, Input Subsystem, and Power Subsystem.

- **<u>Central Controller: ESP32-WROOM-32D</u>:**The key component of the console is the ESP32 Development Board. It acts as the central processing unit, running all game logic, handling Input/Output (I/O) operations, and managing the display buffer. Its dual-core architecture allows for efficient multitasking, which is crucial for maintaining high frame rates during gameplay.

Figure 1: ESP-32

- **<u>Output Subsystem: OLED Display</u>:** For visual output, the system utilizes a **1.3-inch OLED display**.
    - **Protocol:** It communicates with the ESP32 via the **I2C protocol**, using the SDA (GPIO 21) and SCL (GPIO 22) pins.
    - **Function:** It renders monochrome graphics, game states, and menu interfaces. OLED technology was chosen for its high contrast and low power consumption suitable for battery-operated devices.

Figure 2: OLED

- **<u>Input Subsystem: Joystick and Buttons:</u>** The console features a hybrid input system:
    - **Analog Joystick:** This provides primary directional control. It functions as two potentiometers, sending analog voltage signals to the ESP32's ADC pins. Horizontal movement (VRx) is mapped to GPIO 34, and vertical movement (VRy) is mapped to GPIO 35.
    - **Push Buttons:** Three tactile buttons are integrated for "Home," "Back," and "Action" commands. These allow for a full navigation system within the console's software menus.

Figure 3: Joystick

- **<u>Power Subsystem & Chassis:</u>** To ensure true portability, a dedicated power circuit was designed:
    - **Power Source:** Two Li-ion battery cells connected in parallel, providing approximately **1800mAh** of capacity.

- **Regulation:** An **AMS1117-3.3V** Low-Dropout (LDO) regulator steps down the battery voltage to a clean 3.3V required by the ESP32.
- **Charging:** An integrated TP4056 module with a Type-C port allows for convenient recharging.

Figure 4: Li battery with TP4056

- **Chassis:** The physical body uses a custom design with a rigid Vero board base and laser-cut acrylic faceplates, assembled with metal spacers to protect internal components .

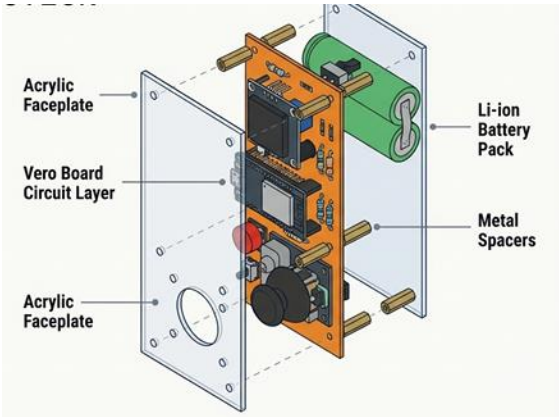

Figure 5: Hardware Assembly

Block Diagram and Circuit Diagram:

The figure below illustrates the block diagram of the console, categorized into four distinct functional subsystems: Inputs, Processing, Outputs, and Power.

The data flow begins at the Input Subsystem, where the analog joystick and push buttons transmit user commands to the central controller. The ESP32-WROOM-32 acts as the brain of the system, processing these signals to execute game logic and update the system state. The processed data is then transmitted to the Output Subsystem, specifically the 1.3-inch OLED display, via the I^2C protocol for visual feedback . A centralized Power Subsystem ensures stable voltage distribution to all three active blocks, ensuring reliable operation.

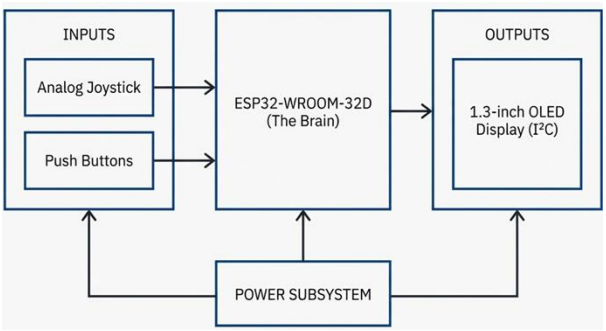

Figure 6: Block Diagram

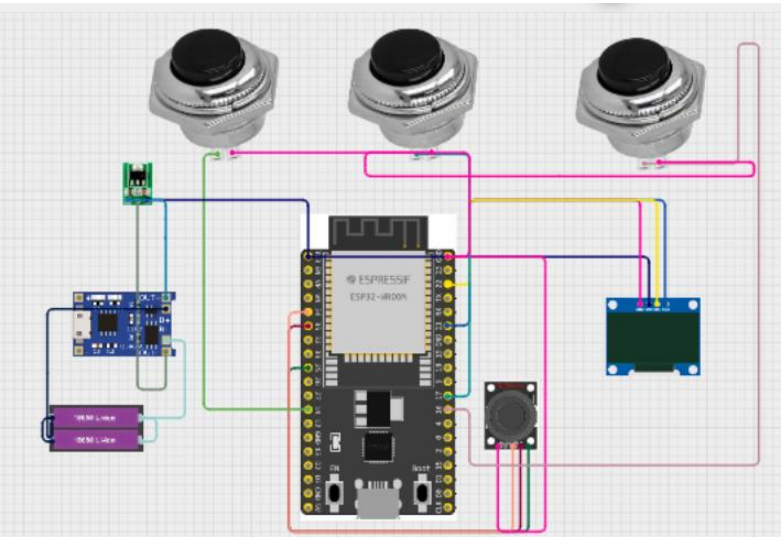

Figure 7: Circuit Diagram

4. Software Integration

The software transforms the hardware into a cohesive console, evolving from a simple loop to a modular architecture.

- **<u>Development Environment and Libraries</u>:** The firmware is developed using the Arduino IDE, using specific libraries to handle low level hardware communication, allowing the main code to focus on game logic.
    - **Graphic Libraries**: These act as the graphics driver. They provide the coordinate system (x, y) needed to draw pixels, lines, and bitmaps (like the snake body or Flappy Bird sprite) on the display.
    - **Wire.h**: This standard library manages the I^2C protocol communication. It sets the clock speed and data transmission between the ESP32 and the OLED .
    - **EEPROM Library**: This is used for non-volatile memory management. It allows the code to write and read high scores to the ESP32's flash memory, ensuring data persists even after power is lost.

- **<u>Finite State Machine (FSM) Architecture</u>:** To manage multiple screens (Menu, Snake, Flappy Bird, Calculator) without the confusion in the code, **Finite State Machine** is implemented.

    - **Implementation:** The code uses a global variable (likely Enum Game State) that tracks the current mode.
    - **Logic:** inside the main loop(), a switch...case structure checks this variable.

        ⇒ *If State = MENU:* It runs the drawMenu() and checkMenuInput() functions.
        ⇒ *If State = SNAKE:* It runs the updateSnake() and drawSnake() functions.
        ⇒ *If State = CALCULATOR:* It switches to the grid-based input logic. This configuration ensures that the joystick only moves the snake when the game is running, and only changes menu items when in the menu.

- **<u>Input Handling and Debouncing</u>:** The raw signals from the push buttons and joystick are noisy and can cause glitches (like registering two clicks instead of one).
    - **Joystick Logic:** The code reads analog values (0-4095) from pins **GPIO 34 (VRx)** and **GPIO 35 (VRy)**.
    - Calibration: The code interprets the center value (approx. 2048) as "neutral." Values below a threshold (e.g., < 1000) register as LEFT/UP, and above a threshold (e.g., > 3000) register as RIGHT/DOWN.
    - **Button Debouncing:** Software debouncing is configured for the discrete buttons (Home, Back, Action). The code checks the button state and then waits a few milliseconds (e.g., 50ms) before confirming the press, ensuring a clean "HIGH" or "LOW" signal is processed.
- **<u>Libraries and Graphics</u>:** The display update logic follows a Clear-Draw-Display cycle to prevent screen flickering:
    - **Clear Buffer:** At the start of every frame, the display buffer is wiped clean.
    - **Calculate Positions:** The game logic calculates the new coordinates of game objects (e.g., moving the snake head one pixel forward).
    - **Draw Frame:** The code draws the new state to the internal memory buffer.
    - **Display:** Finally, the display.display() command sends the entire buffer to the OLED screen via I^2C.
- **<u>EEPROM</u>:** The configuration for high scores involves direct memory mapping:

    - **Reading:** On startup, the setup() function reads specific memory addresses in the EEPROM to retrieve the last saved high scores.

    - **Writing:** When a "Game Over" event occurs, the code compares the current score with the stored value. If the new score is higher, it overwrites the value at that specific memory address.

5. Gaming Performance

The console successfully runs multiple arcade games and applications with smooth frame rates:

- **Snake:** Features multiple difficulty modes (Easy, Normal, Hard) and high-score tracking.
- **Flappy Bird:** A side-scrolling game requiring precise timing.
- **Ping Pong:** Features a single-player mode against a smart AI opponent.

Figure 8:Games Menu

- **Calculator:** A functional grid-based calculator.

Figure 9: Calculator

- **Stopwatch:** A precise timer with start/stop/reset functions.

Figure 10: Stopwatch

- **ESP-NOW RC Control:** Demonstrates wireless capabilities by allowing the console to act as a remote control for other ESP32 devices.

Figure 11: ESP NOW Menu

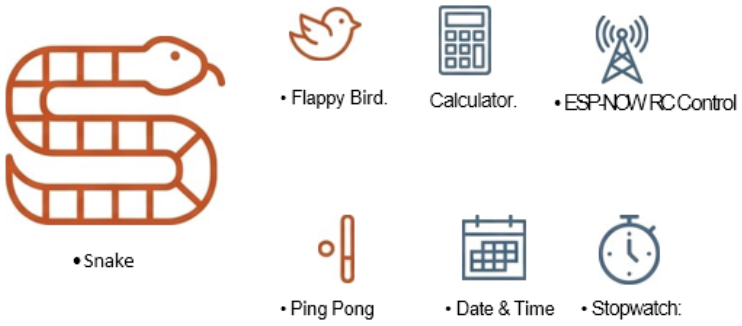

Figure 12: Total Applications

6. Conclusion

The Handheld ESP32 Gamepad has been successfully completed with full functionality. The project demonstrates the effective integration of the ESP32-WROOM-32D microcontroller with external components, including a 1.3-inch OLED display and analog joystick input . By implementing a stable software architecture based on a Finite State Machine, the device evolved from a single-purpose 'Snake' game into a multi-functional console capable of running arcade games (Snake, Flappy Bird, Ping Pong) and utility applications (Calculator, Stopwatch) .